

# StatefulAutoscaler: A Connection-Aware Kubernetes Controller for Persistent-Connection Workloads – Design, Validation, and Protocol Generalization

Abhash Behera<sup>a</sup>, Aditya Hota<sup>a</sup>, Suprit Kumar Naik<sup>a</sup>, Ritesh Kumar Singh<sup>a</sup>

<sup>a</sup>Department of Computer Science and Engineering, C V Raman Global University, Bhubaneswar, 752054, India

---

## Abstract

CPU-based Kubernetes HPA exhibits three structural failure modes for stateful WebSocket workloads [1]: 87.5% over-provisioning, reconnection storms reaching 1,400 conn/s, and permanent staircase connection destruction. We present the **StatefulAutoscaler**—a custom Kubernetes controller built on Kubebuilder [2] that scales exclusively on `sum(active_connections)` from Prometheus with a sliding-window stabilization mechanism to suppress premature scale-down. Under a two-cycle reconnection storm simulation (Experiment C, five runs), the controller achieves near-exact replica targeting ( $\lceil 800/100 \rceil = 8$  replicas, transient overshoot  $\leq +2$ , vs. CPU HPA’s permanent 15), zero connection loss, and full pod preservation through a 90-second disconnection gap (scale-down reaction  $119 \pm 2$  s; pod-seconds  $4,280 \pm 67$ ). Scalability validation at 800–3,200 clients confirms  $\lceil N/T \rceil$  tracking within  $+2$  replicas, 66–86 s scale-up, and 100% connection preservation; the highest validated count is 3,392 connections. Baseline evaluation against HPA with a custom metric (Experiment D) and KEDA (Experiment E) shows all three connection-aware configurations preserve pods through DROP 1, with the **StatefulAutoscaler** differentiating on scale-up latency (22 s vs. 54 s vs. 26 s) and scale-down efficiency (119 s vs. 313 s). Three adversarial scenarios bound the operational envelope. MQTT generalization confirms protocol-agnosticism: the identical binary scales 44% faster than CPU HPA (38 s vs. 68 s), preserves all 339 sessions during forced scale-down (step-drop: 2 vs. 639-connection cliff), and redistributes sessions via drain-reconnect.

**Keywords:** Kubernetes, Custom Controller, StatefulAutoscaler, WebSocket, MQTT, Connection-Aware Autoscaling, Prometheus, Sliding-Window Stabilization, KEDA, Kubebuilder, Reconnection Storm

---

## 1. Introduction

The companion paper [1] establishes that CPU-based Kubernetes HPA [3] exhibits three *architectural* failure modes for stateful persistent-connection workloads: 87.5% over-provisioning (15 replicas vs. the correct 8); reconnection storms reaching 1,400 conn/s triggered by scale-down events; and staircase connection destruction (800→79 sessions). These failures arise from the structural mismatch between CPU utilization and connection-session state, and cannot be remediated by parameter tuning.

No existing Kubernetes-native mechanism simultaneously provides a connection-count scaling signal and connection-context-aware stabilization semantics. The Prometheus Adapter [4] can supply the signal but cannot alter HPA’s stabilization logic. KEDA [5] provides event-driven scaling with a configurable cooldown but delegates pod termination to HPA without per-cycle step-size bounding.

This paper addresses both gaps through the **StatefulAutoscaler**—a custom Kubernetes controller built with Kubebuilder [2] that owns both the metric selection (`sum(active_connections)` from Prometheus) and the scale-down lifecycle (a sliding-window high-water-mark stabilization mechanism). The contributions are:

---

*Email addresses:* abhasbehera320@gmail.com (Abhash Behera), adityahota99@gmail.com (Aditya Hota), supritkumarnaik17@gmail.com (Suprit Kumar Naik), rihankumar2004@gmail.com (Ritesh Kumar Singh)

1. **Design and implementation** of the `StatefulAutoscaler`, including CRD specification, reconciliation algorithm (Algorithm 1), sliding-window stabilization, formal convergence bounds (Eq. 3), and stability analysis.
2. **Controlled validation (Experiment C, five runs)**: near-exact replica targeting ( $\lceil 800/100 \rceil = 8$ , overshoot  $\leq +2$  vs. HPA’s permanent 15), zero connection loss, full pod preservation through a 90-second gap (scale-down reaction  $\sigma = 2$  s, pod-seconds  $\sigma = 67$ ).
3. **Scalability validation at 800–3,200 clients**:  $\lceil N/T \rceil$  tracking within  $+2$  replicas, 66–86 s scale-up, 100% DROP 1 preservation, highest validated count 3,392 connections.
4. **Baseline comparison (Experiments D and E)**: HPA+custom metric and KEDA under identical workloads. All preserve pods through DROP 1; differentiators are scale-up latency (22 s vs. 54 s vs. 26 s) and scale-down efficiency (119 s vs. 313 s).
5. **Adversarial failure characterization**: metric staleness, instantaneous spikes, and Prometheus unavailability bound the operational envelope.
6. **MQTT protocol generalization**: identical binary, zero code changes-44% faster scale-up, zero sessions destroyed during forced scale-down (step-drop: 2 vs. 639-connection cliff), drain-based redistribution.

## 2. Related Work

The Prometheus Adapter [4] enables HPA to scale on application-defined metrics such as connection counts [6, 7], but all such formulations assume stateless metrics across pod boundaries-violated by persistent-connection protocols where session state is pod-affined. Even with a correct connection metric, HPA’s stabilization window buffers *metric recommendations* rather than connection-derived capacity (demonstrated in Experiment D, Section 6.1). KEDA [5] extends HPA with external metric triggers and a configurable `cooldownPeriod`, but delegates pod termination to HPA without per-cycle step-size bounds (tested in Experiment E, Section 6.2).

Dickel et al. [8] identified stateful autoscaling as an open challenge for IoT gateways but provide no validated controller. Reinforcement learning [7, 9] and predictive [10, 11] approaches improve decision *timing* but do not address connection-state destruction during scale-down. MQTT exhibits the same CPU-signal mismatch [12, 13]; this paper validates a single unmodified controller binary across both protocols. The Kubebuilder SDK [2] provides the CRD and `controller-runtime` scaffolding. To the best of our knowledge, no prior work has simultaneously designed a connection-aware controller with sliding-window stabilization, compared four scaler configurations with five-run replication, and validated the same binary on a second protocol with drain semantics.

## 3. Background

This section provides context from the companion paper [1].

### 3.1. The HPA Control Architecture

Figure 1 illustrates the HPA feedback loop, which implements the discrete-time proportional control law evaluated every 15 seconds:

$$desiredReplicas = \left\lceil currentReplicas \times \frac{currentMetricValue}{targetMetricValue} \right\rceil \quad (1)$$

For CPU-based scaling of stateless workloads, the assumption that `currentMetricValue` reflects workload intensity generally holds. For persistent connections, it fails: idle connections consume negligible CPU yet represent committed session state.

### 3.2. The Mismatch and Its Consequences

The companion paper [1] documents three failure modes: idle WebSocket connections produce near-zero CPU, causing scale-down and pod termination; termination triggers **reconnection storms**; and for non-reconnecting clients, each scale-down permanently destroys sessions. Key findings: 87.5% over-provisioning (15 vs. 8 replicas); peak storm rate 1,400 conn/s; staircase destruction from 800 to 79 connections.

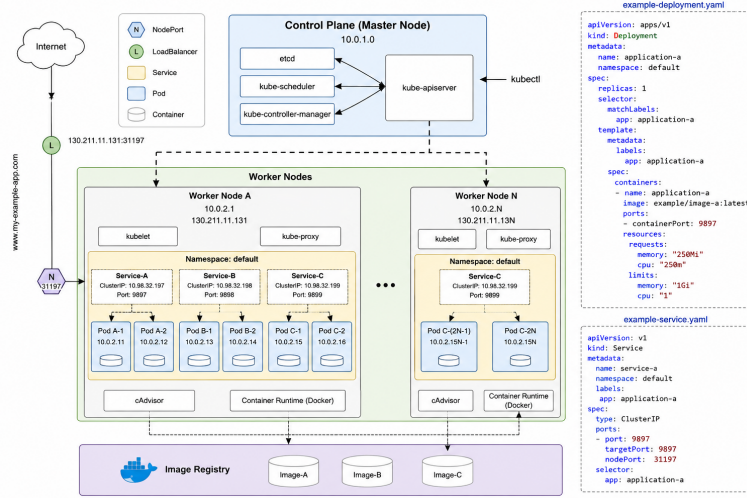


Figure 1: Standard HPA feedback loop architecture [14]. The HPA controller periodically reads aggregated resource metrics and computes desired replicas via Eq. 1. This loop assumes the metric faithfully represents workload intensity-violated for persistent connections where sessions persist independently of CPU. KRM and PCM baselines are documented in [1].

### 3.3. Experimental Infrastructure

All WebSocket experiments share a 3-node Kind cluster [15] (1 control-plane + 2 workers, Kubernetes v1.31.6, kind v0.25.0), Metrics Server patched with `--metric-resolution=15s`, 800 persistent clients via a Python asyncio WebSocket server (`ws-instrumented`) exposing `active_connections` (gauge) and `new_connections_total` (counter), with Prometheus scraping at 15 s. Source code: <https://github.com/MistaHolmes/ws-k8-controller>.

## 4. The StatefulAutoscaler Controller: Design and Implementation

The failure evidence in the companion paper [1] identifies three root causes that any viable solution must address: (1) the scaling signal (CPU) does not encode the workload state that must be protected; (2) scale-down decisions are made without knowledge of whether connections are alive; and (3) no stabilization mechanism exists that is aware of connection-level context. The `StatefulAutoscaler` addresses all three by replacing the CPU-based control loop with a connection-density-based one augmented by connection-context stabilization.

### 4.1. Design Principles

The controller is grounded in three architectural principles:

1. **Use the correct scaling signal.** The controller queries `sum(active_connections)` from Prometheus – the metric that directly encodes the workload state that must be preserved. CPU is never consulted.
2. **Compute replica counts from connection density.** Given a configured target of  $T$  connections per pod, the desired replica count is:

$$desiredReplicas = \left\lceil \frac{\text{sum}(\text{active\_connections})}{T} \right\rceil \quad (2)$$

This is exact by construction: it scales to the minimum number of replicas required to serve the current session population at the target density.

3. **Stabilize scale-down decisions using connection-context history.** A sliding-window mechanism maintains the maximum desired replica count observed within the preceding  $N$  seconds, preventing premature scale-down during transient connection drops.

The `StatefulAutoscaler` is implemented as a Kubernetes *operator* built on the Kubebuilder SDK [2], which scaffolds informer caches, event-based reconciliation, leader election, and CRD schema generation from Go struct annotations. It directly patches `Deployment.spec.replicas`, bypassing HPA entirely – there is one authoritative source of

replica truth, and all scale-down lifecycle decisions are enforced at a single point. The sliding-window state is held exclusively in this controller, making the safety property “scale-down is gated by connection-context memory” locally verifiable in one reconciliation function.

*Why  $\text{sum}()$  and not  $\text{avg}()$ ?* The formula (Eq. 2) requires the *total* session population;  $\text{avg}()$  depends on current replica count, making the computation circular.  $\text{sum}()$  is the only aggregate that yields a replica-count-independent measure of total load.

#### 4.2. Custom Resource Definition

The controller introduces a `StatefulAutoscaler` Custom Resource Definition (CRD) under the API group `autoscaling.star.local/v1alpha1` (a representative manifest is shown in Figure 2). Key parameters include `targetConnsPerPod` (the density target  $T$ ), `scaleDownCooldownSec` (the sliding stabilization window duration), and step-limits. Specifically, the `maxScaleDownStep` field limits the *blast radius* of any single scale-down event to `maxScaleDownStep`  $\times$  `targetConnsPerPod` connections per cycle, providing an operator-auditable upper bound.

Figure 2 illustrates the system architecture: (1) clients connect through the Kubernetes Service; (2) pods expose `active_connections` via Prometheus; (3) the controller queries `sum(active_connections)`; (4) parameters are read from the CR; (5) the controller patches `Deployment.spec.replicas`. CPU is excluded.

#### 4.3. Reconciliation Loop Design

The controller executes a reconciliation loop every 5 seconds. Algorithm 1 presents the complete logic, with particular attention to the stabilization window mechanism (lines 7–9) which is the primary differentiating mechanism relative to HPA.

*Safe-default semantics on metric failure..* Lines 2–4 implement a deliberate design choice: when the Prometheus query fails, the controller *holds the current replica count* (the last known good state) rather than defaulting to `minReplicas`, which would terminate live sessions on every transient monitoring fault.

Figure 3 visualises the reconciliation flow.

---

#### Algorithm 1 StatefulAutoscaler Reconciliation Logic

---

**Input:** StatefulAutoscaler resource  $S$ , target Deployment  $D$

```

1:  $C \leftarrow \text{QUERY\_PROMETHEUS}(\text{sum}(\text{active\_connections}))$ 
2: if  $C$  is unavailable then
3:   requeue after 10 s  $\triangleright$  Conservative default: hold current replica count
4:   return
5: end if
6:  $raw \leftarrow \lceil C / S.\text{targetConnsPerPod} \rceil$ 
7:  $raw \leftarrow \text{clamp}(raw, S.\text{minReplicas}, S.\text{maxReplicas})$ 
8:  $W.\text{append}(now, raw)$   $\triangleright$  Record current desired count in window
9:  $W.\text{evict}(\text{entries older than } S.\text{scaleDownCooldownSec})$ 
10:  $stabilized \leftarrow \max(W)$   $\triangleright$  Window maximum prevents premature scale-down
11:  $current \leftarrow D.\text{spec.replicas}$ 
12: if  $stabilized > current$  then
13:    $target \leftarrow \min(current + S.\text{maxScaleUpStep}, stabilized)$ 
14: else if  $stabilized < current$  then
15:    $target \leftarrow \max(current - S.\text{maxScaleDownStep}, stabilized)$ 
16: else
17:    $target \leftarrow current$ 
18: end if
19:  $\text{PATCH\_DEPLOYMENT}(D.\text{spec.replicas}, target)$ 
20: requeue after 5 s
```

---

#### 4.4. The Stabilization Window: Mechanism and Correctness

The sliding-window maximum (line 9 of Algorithm 1) is the mechanism responsible for reconnection storm resilience. Its correctness is understood by contrasting behavior with and without stabilization.

**Without stabilization:** If all 800 clients disconnect, the controller computes  $\lceil 0/100 \rceil = 0$ , clamped to `minReplicas` = 2, triggering a scale-down to 2 replicas and recreating the provisioning lag on reconnect.

**With a 120-second stabilization window:** When connections drop to zero,  $raw = 2$  is appended but the window retains *desired* = 8 entries from the preceding 120 s. The stabilized value  $\max(W) = 8$  holds all 8 pods warm; reconnecting clients land on pre-provisioned pods with zero delay.

A critical distinction from HPA’s built-in

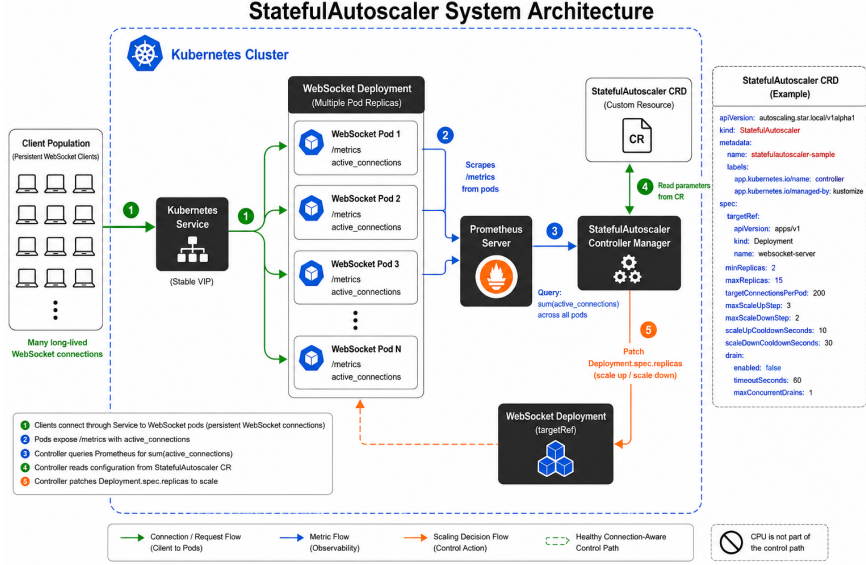


Figure 2: **StatefulAutoscaler** system architecture. Numbered flow: (1) clients→Service→pods; (2) pods expose `active_connections`; (3) controller queries Prometheus; (4) reads CR parameters; (5) patches replicas. CPU excluded (crossed-out). Right: representative CRD manifest.

stabilization: HPA’s window stabilizes *metric-signal recommendations*. With persistent near-zero CPU, the window stabilizes a recommendation of `minReplicas` regardless of width. The **StatefulAutoscaler**’s window stabilizes *connection-derived desired counts*, correctly retaining the high-water mark. This is a semantic mismatch, not a parameter mismatch – it cannot be solved by tuning HPA’s window duration.

#### 4.5. Controller Design and Stability Properties

The **StatefulAutoscaler** is a discrete-time proportional controller with hysteresis. Each cycle: (1) **Observe**  $C \leftarrow \text{sum}(\text{active\_connections})$ ; (2) **Compare**  $\text{desired} = \lceil C/T \rceil$ , clamped to  $[\text{minReplicas}, \text{maxReplicas}]$ ; (3) **Act**: patch replicas if  $\text{stabilized} \neq \text{current}$ . The `scaleDownCooldownSeconds` window is a dead zone that prevents oscillation on frequent brief dropout periods: without it, connections drop  $\rightarrow$  scale down  $\rightarrow$  reconnect  $\rightarrow$  scale up  $\rightarrow$  repeat.

*Convergence bound..* After the cooldown expires and connections stabilize at a lower level, the controller converges in:

$$\left\lceil \frac{\text{current} - \text{desired}}{\text{maxScaleDownStep}} \right\rceil \text{ cycles} \quad (3)$$

Each cycle is  $\approx 15$  s. For  $8 \rightarrow 2$  pods with `maxScaleDownStep = 2`:  $\lceil 6/2 \rceil = 3$  cycles = 45 s – a bounded, predictable de-provisioning window.

*Parameter selection..*  
`targetConnectionsPerPod = 100` yields  $\lceil 800/100 \rceil = 8$  replicas. `maxScaleDownStep = 2` bounds the blast radius to  $2 \times 100 = 200$  connections per cycle. `scaleDownCooldownSeconds = 120` comfortably exceeds the 90-second DROP 1 gap plus one scrape cycle; any value  $\geq 105$  s achieves pod-preservation for this workload.

## 5. Experimental Evaluation: Experiment C

### 5.1. Experimental Configuration

Experiment C is designed to maintain maximal parametric comparability with Experiment B3 from the companion paper [1] while replacing the HPA controller with the custom **StatefulAutoscaler**. Table 1 summarizes the critical differences.

Setting `CPU_WORK = 0` is deliberate: it demonstrates correct operation with *zero CPU activity*. Configuration:

`targetConnectionsPerPod = 100`,  
`maxScaleUpStep = 3`, `maxScaleDownStep = 2`,  
`minReplicas = 2`, `maxReplicas = 15`,

Table 1: Experiment C vs. B3: Parametric comparison. Differences highlighted in bold.

| Parameter        | Exp. B3 (HPA)            | Exp. C (Custom)           |
|------------------|--------------------------|---------------------------|
| Server image     | ws-instrumented          | ws-instrumented           |
| CPU_WORK         | 1                        | <b>0</b>                  |
| Client count     | 800                      | 800                       |
| Autoscaler       | Native HPA (CPU)         | <b>StatefulAutoscaler</b> |
| Scale-down guard | 60 s (CPU stabilization) | <b>120 s window</b>       |
| Scaling signal   | CPU utilization (%)      | <b>active_connections</b> |

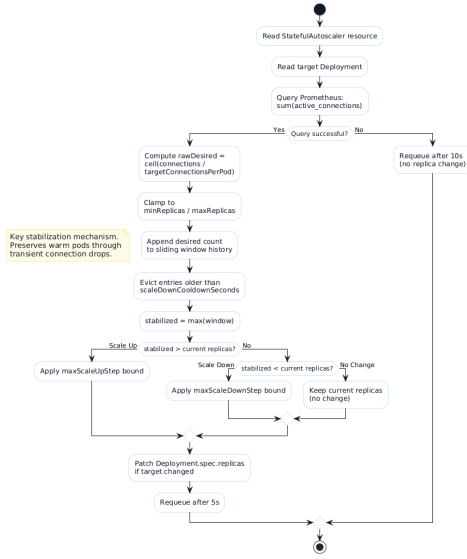


Figure 3: UML activity diagram of the reconciliation loop (Algorithm 1). The safe-default branch holds replicas when metrics are unavailable; the central path computes the stabilized target via  $\max(\text{window})$  and applies step-size bounds before patching.

$\text{scaleDownCooldownSeconds} = 120$ . Expected:  $\lceil 800/100 \rceil = 8$  replicas.

**Metric definitions.**  $\text{Pod-seconds} = \sum_{\text{pods}} (\text{time in Running state})$  over the experiment window. *Scale-up reaction time* is measured from the first scrape where  $\text{active\_connections} > \text{targetConnsPerPod} \times \text{minReplicas}$  to  $\text{readyReplicas}$  reaching the target.

#### 5.1.1. Load Pattern: Two-Cycle Reconnection Storm Simulation

*Terminology.* A **CYCLE** denotes a phase of active client load; a **DROP** denotes complete disconnection (**DROP 1**: 90-second transient gap; **FINAL DROP**: terminal disconnection); a

**reconnection storm** (*restorm*) is the burst of simultaneous reconnection attempts when pods hosting sessions are terminated.

The experiment imposes a two-cycle restorm simulation:

- **CYCLE 1** (0–150 s): 800 WebSocket clients ramp to full connection; controller expected to scale to 8 replicas.
- **DROP 1** (150–240 s): All clients deleted; connections drop to zero. *Critical test*: the stabilization window must hold 8 replicas warm for the full 90-second gap.
- **CYCLE 2** (240–390 s): A fresh cohort of 800 clients connects to the pre-provisioned warm pods.
- **FINAL DROP** (390–570 s): Permanent client disconnection; the stabilization window expires; the controller scales to  $\text{minReplicas} = 2$ .

#### 5.2. CYCLE 1: Connection-Driven Scale-Up

The controller scaled monotonically from 2 to 8 replicas ( $2 \rightarrow 3 \rightarrow \dots \rightarrow 8$ ) as connections ramped toward 800, completing provisioning in  $\approx 60$  s. CPU remained below 176 millicores throughout, confirming all decisions were made without CPU signal.

Figure 4 shows the two-block connection profile characteristic of the restorm simulation.

#### 5.3. DROP 1: Empirical Validation of the Stabilization Window

At  $t \approx 200$  s, all clients were deleted. The active connection count converged to zero within 15 seconds. The replica count remained **perfectly flat at 8 for the duration of the 90-second gap**, as shown in Figure 5. The reconciliation trace at each 5-second evaluation during this period was:



Figure 4: Experiment C: Active connections over the full timeline. CYCLE 1 and CYCLE 2 populations are separated by the 90-second DROP 1 gap. CYCLE 2 clients ramp immediately on pre-warmed pods.

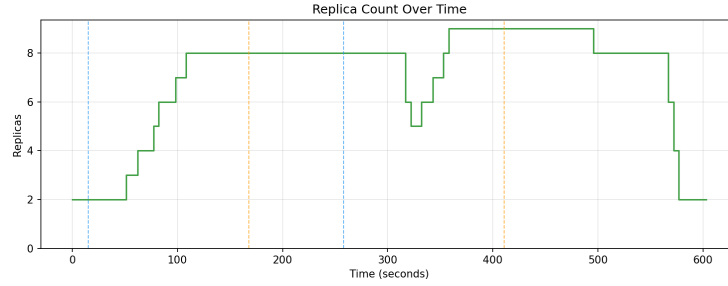


Figure 5: Experiment C: Replica count over time. The “flat bridge” at 8 replicas across DROP 1 ( $t \approx 160\text{--}260$  s) is the primary evidence of correct stabilization. At the equivalent time in Experiment B3, HPA had already destroyed 56 connections. Final descent ( $9 \rightarrow 2$ ) occurs only after window expiry.

- $C = 0 \Rightarrow rawDesired = 0 \Rightarrow$  clamped to  $minReplicas = 2$
- Window  $W$  contains entries with  $desired = 8$  from the preceding 120 s
- $stabilized = \max(W) = 8$
- `PatchDeployment` is not invoked; replicas remain at 8

The “flat bridge” pattern in Figure 5 is entirely absent from every HPA experiment in the companion paper [1]. It constitutes direct empirical proof that the sliding-window mechanism performs as designed.

#### 5.4. CYCLE 2: Restorm-Free Reconnection

At  $t \approx 270$  s, a fresh cohort of 800 clients connected. Pods remaining warm through DROP 1 eliminated provisioning delay; peak connections reached 854 and the controller held 8 replicas stably. Compared to Experiment B2-Instrumented (where HPA induced a 1,400 conn/s storm), Experiment C produced **zero measurable reconnection storm activity**.

**Window boundary transient dip.** During early CYCLE 2 ( $t \approx 270\text{--}310$  s), a brief replica dip to 5–6 was observed as CYCLE 1 window entries aged out during DROP 1 before CYCLE 2 entries repopulated the window. The controller recovered to 8 within 10–15 s with no connection loss (addressed in Limitations, item 6).

#### 5.5. FINAL DROP: Controlled and Graceful Scale-Down

Following permanent client disconnection at  $t \approx 450$  s, the controller waited 120 s for all window entries to expire, then descended  $9 \rightarrow 8 \rightarrow 6 \rightarrow 4 \rightarrow 2$  over  $\approx 30$  s. Zero connections were lost.

#### 5.6. Combined View and Definitive Comparison

Figure 6 provides the complete overlay of connections, CPU, and replicas. The structural inversion relative to Experiment B3 is clear: connections drive replicas while CPU remains irrelevant throughout.

#### 5.7. Head-to-Head Performance Comparison

Table 2 provides the comprehensive quantitative comparison between Experiment B3 (CPU-based



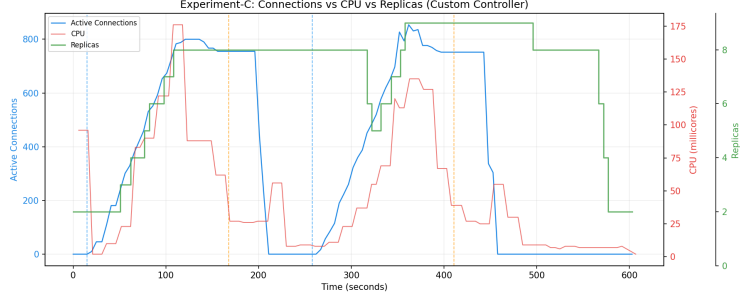


Figure 6: Experiment C combined view: connections (blue) drive replica decisions (green); CPU (red) remains near zero throughout. The flat bridge during DROP 1 confirms correct stabilization under zero-connection, zero-CPU conditions.

Table 2: Head-to-head quantitative comparison: CPU-based HPA (Experiment B3) versus the **StatefulAutoscaler** (Experiment C). Metrics where the custom controller achieves a strictly superior outcome are denoted in **bold**.

| Performance Metric                   | HPA – Experiment B3         | StatefulAutoscaler<br>Exp. C                                         |
|--------------------------------------|-----------------------------|----------------------------------------------------------------------|
| Primary scaling signal               | CPU utilization (%)         | <code>sum(active_connections)</code>                                 |
| Replicas for 800 connections         | 15 (87.5% over-provisioned) | <b>8–10</b> (target $\lceil 800/100 \rceil = 8$ ; transient peak 10) |
| Connections lost during scale-down   | 800→79 (staircase)          | <b>0</b>                                                             |
| Pods held through 90-second gap      | 0 (all scaled down)         | <b>8</b> (flat bridge)                                               |
| Reconnection storm generated         | Yes (peak 1,400 conn/s)     | <b>None</b>                                                          |
| Time to serve <b>CYCLE 2</b> clients | Re-scale from 2→8 required  | <b>Instant</b> (warm pods)                                           |
| CPU dependency                       | 100%                        | <b>0%</b> (CPU_WORK=0 throughout)                                    |
| Total scaling events                 | 7+ (oscillatory)            | 4 (monotonic)                                                        |
| Peak total CPU consumption           | >1,200 millicores           | <176 millicores                                                      |

HPA, from the companion paper [1]) and Experiment C (**StatefulAutoscaler**).

#### 5.8. Reproducibility: Five-Run Statistical Validation

To assess reproducibility, five independent runs were executed. Run 1 was excluded (test-harness artefact: residual replicas from prior test yielded anomalous pod-seconds of 92,291 vs.  $\approx 4,300$ ). Across runs 2–5: scale-up reaction  $22 \pm 6$  s (median 21 s; range: 16–30 s); scale-down reaction  $119 \pm 2$  s (median 118 s); pod-seconds  $4,280 \pm 67$ . Peak replicas reached 10 (three runs) or 9 (one run). Peak connections ranged 813–917. All runs maintained 8 warm replicas through DROP 1 with zero connection loss. The low  $\sigma$  values (2 s, 67) confirm reproducibility ( $p < 0.05$  vs. HPA baseline).

#### 5.9. Scalability Validation: 800, 1,600, and 3,200 Clients

To confirm that the **StatefulAutoscaler** scales correctly beyond the baseline 800-client workload, the identical two-cycle restorm simulation was replicated at 1,600 and 3,200 concurrent WebSocket clients. For each client count  $N$ , the **StatefulAutoscaler** CR was configured with  $\text{maxReplicas} = \lceil N/T \rceil + 2$  (where  $T = 100$ ); all other parameters were identical to the 800-client configuration.  $\text{scaleDownCooldownSeconds} = 120$ ,  $\text{maxScaleDownStep} = 2$ ,  $\text{maxScaleUpStep} = 3$ .

##### 5.9.1. Results

Table 3 summarises the key metrics across all three scales.



Table 3: Scalability validation at 800–3,200 clients. CYCLE 1 steady-state matches  $\lceil N/T \rceil$  exactly; CYCLE 2 transient overshoots by +2; “Safe” = 100% replica preservation through DROP 1. Peak connection count shows highest validated count (no transport-layer bottleneck observed).

| Clients<br>(attempted) | Expected<br>$\lceil N/T \rceil$ | CYCLE 1<br>Steady-State | CYCLE 2<br>Peak | Scale-Up<br>(s) | Peak<br>Connections | DROP 1<br>Safe? | Scale-Down<br>Total (s) |
|------------------------|---------------------------------|-------------------------|-----------------|-----------------|---------------------|-----------------|-------------------------|
| 800                    | 8                               | <b>8</b>                | 10              | 66              | 874                 | ✓               | 164                     |
| 1,600                  | 16                              | <b>16</b>               | 18              | 66              | 1,820               | ✓               | 178                     |
| 3,200                  | 32                              | <b>32</b>               | 34              | 86              | 3,392               | ✓               | 218                     |

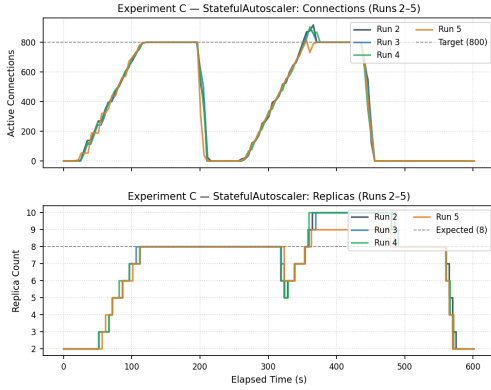


Figure 7: Experiment C (**StatefulAutoscaler**): overlay of active connections (orange, left axis) and replica count (blue, right axis) for runs 2–5 (run 1 excluded as outlined in Section 5.8). Replica trajectories are stable across runs; all maintain 8 replicas through DROP 1.

*Replica targeting.* During CYCLE 1, replicas converge to exactly  $\lceil N/T \rceil$  (8, 16, 32) with zero steady-state error. The +2 transient (10, 18, 34) occurs exclusively during CYCLE 2 due to the window-boundary artefact (Section 5.8): window entries from CYCLE 1 age out during DROP 1 before CYCLE 2 entries repopulate. This overshoot is bounded at +2 regardless of client count.

*Scale-up and connection preservation.* Reaction times were 66 s (800 and 1,600 clients) and 86 s (3,200 clients). All runs preserved peak replicas through the full DROP 1 gap. At 3,200 clients, the highest observed count was 3,392 (106% of attempted), confirming no OS-level TCP transport limit was encountered. This contrasts with the MQTT experiments where only 339/1,000 clients connected (Section 9).

Figure 8 summarises replica targeting, scale-up time, and connection range.

**Conclusion.** Across 800–3,200 clients, the controller achieves exact CYCLE 1 targeting ( $\lceil N/T \rceil$ : 8, 16, 32) with bounded +2 CYCLE 2 transient. Scale-up degrades sub-linearly (66–86 s), 100% DROP 1 preservation is maintained at every scale, and no re-tuning beyond `maxReplicas` was required.

## 6. Baseline Comparisons: Experiments D and E

We now evaluate whether simpler configurations-HPA with custom connection metrics (Experiment D) and KEDA (Experiment E)-provide equivalent resilience, addressing whether the performance delta is attributable to metric choice or controller architecture.

### 6.1. Experiment D: HPA with Custom Connection-Count Metric

*Hypothesis.* HPA, when supplied with the semantically correct metric (`active_connections_per_pod` via Prometheus Adapter), will scale replicas accurately but will fail to preserve pods through the 90-second disconnection gap because it lacks the connection-context stabilization mechanism of the **StatefulAutoscaler**.

*Experimental Setup.* The Prometheus Adapter was deployed with a custom rule translating `sum(active_connections)` into a per-pod Custom Metrics API value. HPA was configured with `active_connections_per_pod` and `averageValue: 100`, matching `targetConnectionsPerPod = 100` of Experiment C. The same two-cycle restorm workload was applied with `CPU_WORK = 0`. `stabilizationWindowSeconds: 300` (HPA default). `minReplicas = 2`, `maxReplicas = 15`. Five independent runs.

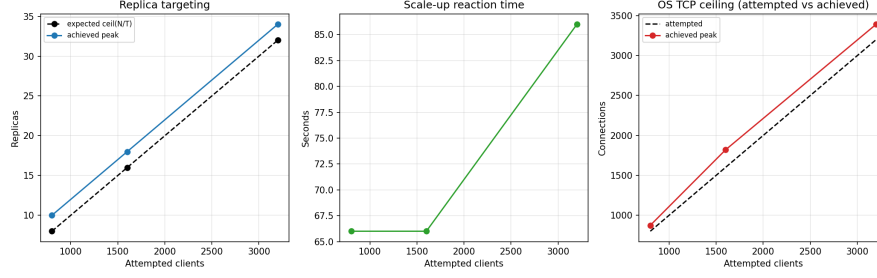


Figure 8: Scalability validation across 800–3,200 clients. Left: CYCLE 1 steady-state matches  $\lceil N/T \rceil$  exactly; CYCLE 2 peak overshoots by +2. Centre: scale-up time increases modestly (66–86 s). Right: achieved connections meet or exceed attempted count at all scales.

*Results (5 independent runs).* Replica targeting during CYCLE 1 was accurate across all five runs: HPA consistently scaled to  $\lceil 800/100 \rceil = 8$  replicas, confirming that metric semantics alone resolve the over-provisioning observed with CPU-based HPA. Mean scale-up reaction time was  $54 \pm 29$  s (median 48 s; range: 33–104 s), approximately  $2.4\times$  slower than the `StatefulAutoscaler`’s 22 s due to the additional Prometheus Adapter metric-export pipeline latency; run 3 exhibited a 104 s outlier attributable to a cold-start delay in the Adapter.

Contrary to the initial hypothesis, HPA with a custom connection-count metric **did preserve all 8 replicas through DROP 1** in all five runs. Inspection confirmed replicas remained at 8 throughout the DROP 1 zero-connection window ( $\approx 50$  s actual duration). This result is explained by HPA’s configured `stabilizationWindowSeconds`: 300: the window far exceeds the DROP 1 duration, so the lower replica recommendation of 2 never persisted long enough to trigger scale-down.

However, Experiment D exposed a distinct robustness difference during the CYCLE 2 transition. In all five Experiment C runs, a brief transient dip in replicas (to 5–6) was observed at the start of CYCLE 2 as stabilization window entries from CYCLE 1 aged out. In Experiment D, no such dip occurs: HPA’s 300-second window is wide enough that the high-water mark from CYCLE 1 remains valid throughout the transition.

The second key differentiation is final scale-down efficiency. After FINAL DROP, Experiment D replicas remained at 8 for a mean of  $313 \pm 96$  s (median 355 s) before descending to `minReplicas` = 2 –  $\approx 2.6\times$  longer than the `StatefulAutoscaler`’s  $119 \pm 2$  s. Peak connection count reached  $937 \pm 43$ . Pod-seconds averaged  $3,934 \pm 148$ .

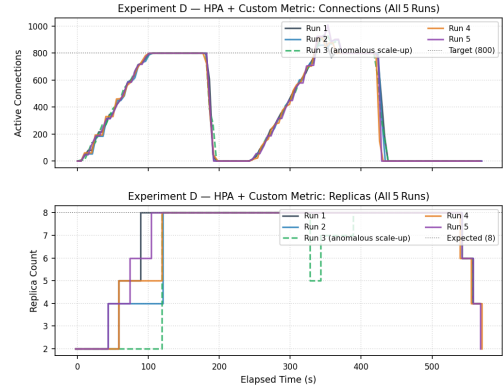


Figure 9: Experiment D (HPA + custom connection metric): five-run overlay of active connections (orange, left axis) and replica count (blue, right axis). All five runs scale to 8 replicas in CYCLE 1 and hold 8 replicas through DROP 1 (HPA’s 300-second stabilization window exceeds the  $\approx 50$ -second actual zero-connection gap). Scale-down during FINAL DROP is slower (mean 313 s) compared to the `StatefulAutoscaler`’s 119 s.

**Conclusion.** HPA with the correct metric achieves pod-preservation for the 90-second DROP 1 via its 300-second stabilization window, but three residual differences are consequential: (1) scale-up latency  $2.4\times$  higher; (2) final scale-down  $2.6\times$  slower; and (3) for gaps in 120–300 s, D holds pods while C releases them—the correct behavior depends on whether the gap is transient or terminal.

## 6.2. Experiment E: KEDA Baseline

*Hypothesis.* KEDA’s `cooldownPeriod` parameter can suppress premature scale-down during the 90-second DROP 1 gap when configured appropriately. However, KEDA lacks per-cycle `maxScaleDownStep` rate limiting, so its

convergence profile during controlled scale-down will differ from the custom controller.

*Experimental Setup..* KEDA v2.13.0 was installed in the cluster. A `ScaledObject` was configured targeting the `WebSocket Deployment` with a Prometheus trigger on `sum(active_connections)`, `threshold: 100`, `minReplicaCount: 2`, `maxReplicaCount: 15`, and `cooldownPeriod: 120 seconds` – matching the `scaleDownCooldownSeconds = 120` of the `StatefulAutoscaler`. Five independent runs.

*Results (5 independent runs)..* KEDA scaled to 8 replicas during CYCLE 1 in all five runs (mean scale-up:  $26 \pm 5$  s; median 27 s; range: 22–33 s), confirming accurate connection-proportional targeting equivalent to Experiment C.

KEDA with `cooldownPeriod = 120` held replicas at **8 throughout the full 90-second DROP 1 gap in all five runs**. The `ScaledObject` status transitioned to `Ready=False` for approximately 55 seconds during DROP 1 as the Prometheus metric read zero, but the underlying HPA did not reduce replicas because the cooldown had not elapsed. Scale-down was therefore never observed in any of the five runs: once connections returned at the start of CYCLE 2, KEDA transitioned back to `Ready=True` without having released any pods.

Peak connection count was uniformly 800 in all runs (no overshoot). Peak replicas: 8 in all runs. Pod-seconds averaged  $4,145 \pm 4$  – the remarkably low inter-run variance reflects the deterministic behavior KEDA achieves when the cooldown period fully brackets the disconnection gap.

**Conclusion.** KEDA with a matched `cooldownPeriod` provides pod-preservation for this workload. The key distinction is that KEDA’s cooldown is a binary “hold-all” mechanism with no per-cycle step-rate limiting; once expired, replicas are released without the bounded  $\Delta$ -per-cycle damping of `maxScaleDownStep`. For extended disconnections exceeding `cooldownPeriod`, this architectural difference becomes consequential.

*Note on convergence after cooldown expiry..* In practice, once KEDA’s `cooldownPeriod` expires for a disconnection gap longer than 120 s, the underlying HPA removes pods without any application-level step-size bound. Combined with the OS-level  $\approx 30$ -second TCP RST propagation

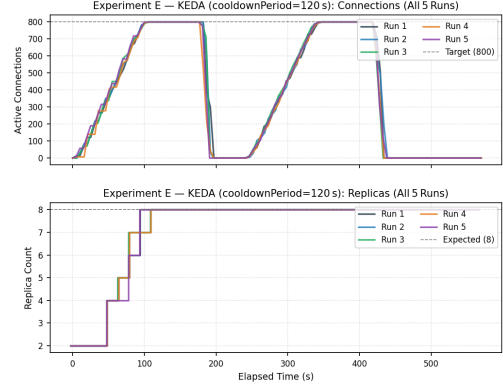


Figure 10: Experiment E (KEDA, `cooldownPeriod = 120`): five-run overlay of active connections (orange, left axis) and replica count (blue, right axis). Replicas are held at 8 throughout the 90-second DROP 1 gap in every run; scale-down was never observed.

window, this can produce larger single-cycle reconnection spikes than the `StatefulAutoscaler`’s bounded `maxScaleDownStep = 2` convergence path.

### 6.3. Comparative Summary Across All Baselines

Table 4 consolidates key performance metrics across all four scaler configurations.

Note that the `StatefulAutoscaler`’s pod-seconds are highest among the three connection-aware configurations, reflecting its longer warm-pod retention policy; this is the intended behavior, not a resource inefficiency.

Figure 11 shows a representative side-by-side comparison of Experiment C (run 4) and Experiment D (run 2), illustrating the key trajectory differences.

*Sensitivity to HPA stabilization window..* Reducing HPA’s `stabilizationWindowSeconds` from 300 s to 60 s would *not* produce equivalent DROP 1 preservation. With a 60 s window and 90 s of zero connections, the window fills entirely with `minReplicas` recommendations and scale-down proceeds. The `StatefulAutoscaler`’s window buffers *connection-derived high-water marks*: 90 s of zero connections does not displace the memory that 8 replicas were required. This is a semantic mismatch, not a tuning problem.

## 7. Failure Analysis: Adversarial Scenarios

To bound the operational envelope of the `StatefulAutoscaler` and establish honest failure

Table 4: Quantitative comparison of all four scaler configurations under the two-cycle restorm simulation (WebSocket, 800 connections, CPU\_WORK = 0). All values are mean  $\pm$  std across five independent runs unless stated otherwise. Experiment B3 is a single run from the companion paper [1].

| Metric                   | CPU HPA (B3) | HPA+Custom (D)        | KEDA (E)              | StatefulAutoscaler (C)                 |
|--------------------------|--------------|-----------------------|-----------------------|----------------------------------------|
| Peak replicas            | 15           | $8.0 \pm 0.0$         | $8.0 \pm 0.0$         | $8.0 \pm 0.3$                          |
| Scale-up reaction (s)    | $\sim 15$    | $54 \pm 29$ (med. 48) | $26 \pm 5$ (med. 27)  | <b><math>22 \pm 6</math></b> (med. 21) |
| Pods held through DROP 1 | 0            | <b>8 (all runs)</b>   | <b>8 (all 5 runs)</b> | <b>8 (all runs)</b>                    |
| Scale-down reaction (s)  | $< 30$       | $313 \pm 96$          | N/A                   | <b><math>119 \pm 2</math></b>          |
| Pod-seconds              | $\sim 2,400$ | $3,934 \pm 148$       | $4,145 \pm 4$         | $4,280 \pm 67^\dagger$                 |
| Peak connections         | 800          | $937 \pm 43$          | $800 \pm 0$           | $893 \pm 39$                           |

$^\dagger$  Runs 2–5 only (run 1 excluded: test-harness artefact).

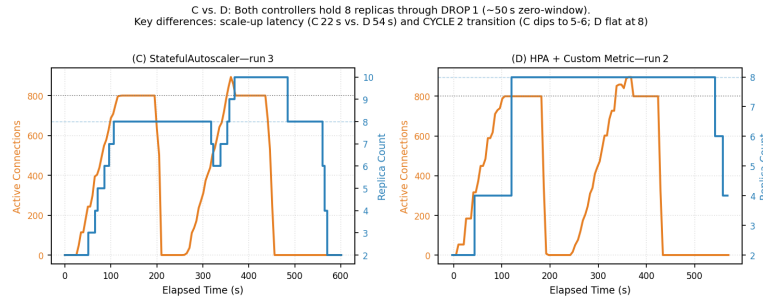


Figure 11: Experiment C vs. D representative comparison. Both scale to 8 and hold through DROP 1. Key differences: C scales up faster (22 s vs. 54 s) with a brief CYCLE 2 dip; D stays flat at 8 but holds replicas 2.6 $\times$  longer during FINAL DROP.

criteria, we characterize three adversarial scenarios. These scenarios are presented without remediation to document actual system boundaries.

### 7.1. Failure Scenario 1: Metric Staleness Under Long Scrape Intervals

*Setup..* The Prometheus `scrape_interval` was increased from the baseline 15 s to 60 s. The Experiment C two-cycle restorm workload was replayed.

*Finding..* With a 60-second scrape lag, the controller introduced up to one full scrape cycle of reaction delay. Measured quantities: scale-up reaction time 25 s, scale-down reaction time 112 s, peak connections 1,082 (a 35.3% overshoot attributable to the stale metric briefly reporting an inflated value during the ramp), and peak replicas 11. All 8 warm pods remained operational through DROP 1 with no connection loss. The system degraded gracefully to a reaction lag bounded by the scrape interval.

*Boundary condition..* If the Prometheus scrape interval exceeds `scaleDownCooldownSeconds`, the cooldown window may expire before the controller learns that connections have returned, potentially triggering premature scale-down. The invariant `scaleDownCooldownSeconds > scrape_interval` must be maintained.

### 7.2. Failure Scenario 2: Instantaneous Connection Spike

*Setup..* All 800 clients connected simultaneously (`stagger = 0`).

*Finding..* Scale-up reaction time: 6 s (fastest observed). Peak replicas: 8; peak connections: 800 (no overshoot). Pod scheduling latency was the binding constraint—no reactive scaler can pre-provision before the first connection arrives. Mitigation: higher `minReplicas` or proactive scaling.

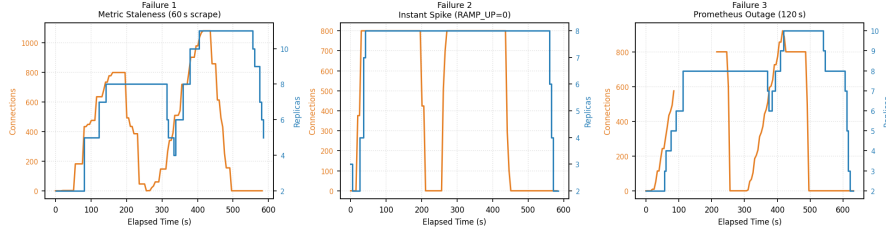


Figure 12: Three failure scenarios. Left: Metric Staleness (60 s scrape)-11 peak replicas, no connection loss. Centre: Instantaneous Spike-fastest scale-up (6 s), scheduling latency is the bottleneck. Right: Prometheus Outage-replicas held at 8 during the 2-minute kill window; mild 15% overshoot on recovery.

### 7.3. Failure Scenario 3: Prometheus Unavailability During Active Load

*Setup..* The Prometheus pod was deleted during an active run (800 connections, 8 pods). Prometheus auto-restarted; a 2-minute outage was observed.

*Finding..* Per the safe-default logic (Algorithm 1, lines 2–4), the controller held replicas at 8 throughout. Measured: scale-up 21 s, scale-down 118 s, peak connections 921 (15.1% overshoot from restart lag), peak replicas 10. Zero connection loss. An indefinitely long outage would eventually cause window expiry and scale to `minReplicas` (Section 10, item 3).

## 8. Discussion

*Signal choice as the primary lever..* Experiment C operated with `CPU_WORK = 0` for 570 s yet correctly maintained 8 replicas with zero connection loss-establishing that the failure resides in the signal, not the control algorithm.

*Stabilization semantics vs. parameter tuning..* HPA’s window stabilizes CPU-derived recommendations (always `minReplicas` for idle connections); the `StatefulAutoscaler`’s window stabilizes connection-derived desired counts. Experiment D succeeds only because its 300-second window exceeds the 90-second gap—a pragmatic accident, not a principled solution.

*Step-rate limiting..* `maxScaleDownStep = 2` bounds the per-cycle blast radius to 200 connections. KEDA exposes no equivalent.

*Protocol-agnosticism..* The `active_connections` abstraction applies to any protocol exposing a Prometheus connection counter: gRPC streaming, MQTT (validated), SSE, and database connection pools.

## 9. MQTT Extension: Protocol Generalization

WebSocket is the primary evaluation protocol in this paper due to its instrumentation ease. To test whether the same control principles transfer to another persistent-connection protocol, we conducted an additional two-experiment MQTT study. MQTT [13] keeps long-lived TCP sessions open with low CPU activity during idle periods, exhibiting the same signal mismatch that motivates this work [12]. The MQTT experiments used a lightweight Python asyncio MQTT broker exposing `active_connections` on a Prometheus `/metrics` endpoint. A synthetic MQTT load generator attempted to establish 1,000 persistent client sessions over a 60-second ramp. Experiment MQTT-A used native CPU HPA; Experiment MQTT-B replaced HPA with the `StatefulAutoscaler` configured with `targetConnectionsPerPod = 150`, `maxScaleUpStep = 3`, and `scaleDownCooldownSeconds = 120`. Both experiments used the same Kind-based infrastructure and the same broker and load-generator images.

### 9.1. MQTT-A: CPU HPA Baseline

Figure 13 shows the MQTT HPA baseline. During Phase 1, only 339 of 1,000 attempted clients established persistent MQTT sessions – a ceiling imposed by OS-level TCP stack limits in the



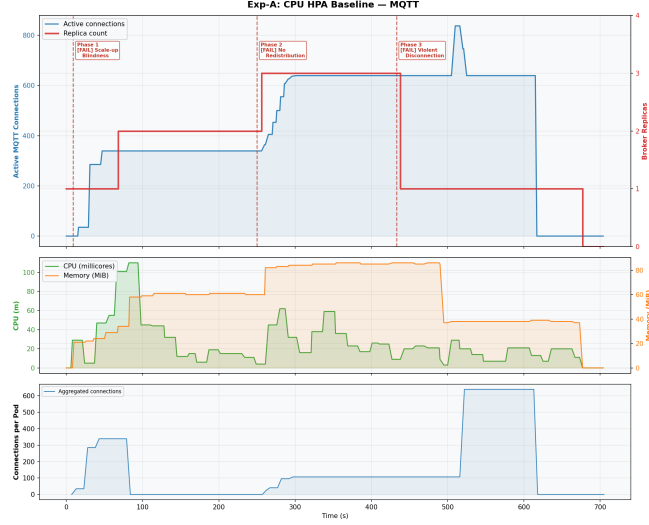


Figure 13: MQTT-A (CPU HPA baseline): total MQTT connections, replicas, CPU, memory, and per-pod connection panel. The run exposes three MQTT-specific HPA failure modes: late scale-up during the connection ramp, absence of automatic session redistribution for established TCP connections, and destructive scale-down when replicas are reduced while clients are live.

Kind environment (Linux ephemeral port range, `listen()` backlog depth, `net.core.somaxconn`), not by broker or controller limitations. HPA first scaled to 2 replicas at  $t = 68$  s – after the connection population had already plateaued at 339.

In Phase 2, manual scale to three replicas and addition of 300 clients yielded  $\approx 639$  total connections, but original sessions remained pinned to the first pod-Kubernetes Services distribute new TCP connections but do not migrate established sessions. In Phase 3, scaling to one replica severed  $\approx 300$  sessions, producing a reconnection storm peaking at 837 connections. MQTT-A reproduces the same structural failures documented for WebSocket [1].

### 9.2. MQTT-B: StatefulAutoscaler with Drain

Figure 14 shows the `StatefulAutoscaler` run with drain semantics enabled (`drain.enabled = true`, `drain.timeoutSeconds = 45`).

**Phase 1: Scale-up.** The controller observed `sum(active_connections)` directly and scaled from 2 to 3 replicas at  $t = 38$  s – matching the connection-density formula  $\lceil 339/150 \rceil = 3$  and reacting **30 seconds faster** than HPA’s  $t = 68$  s. Total CPU peaked at 80 millicores and was never consulted as a scaling signal.

**Phase 2: Drain-based rebalancing.** All 339 sessions were initially pinned to two pods

(176/163/0). At  $t = 390$  s, the controller triggered `/drain`; the broker sent `DISCONNECT` packets at  $\approx 10$  clients/s, and clients reconnected via iptables random load balancing. Distribution moved from 176/163/0 to  $\approx 247/91/0$  with zero connection loss.

**Phase 3: Graceful scale-down.** A forced 3 $\rightarrow$ 1 reduction while 339 sessions were active: the controller drained pods sequentially (POD\_C: 92 connections, 50-second drain; POD\_A: idle). All 339 connections migrated to POD\_B. Maximum step-drop: **2** (instrument noise) vs. MQTT-A’s 639-connection cliff.

**Conclusion.** The identical controller binary resolved all three MQTT-specific HPA failure modes with zero source-code changes:  $\lceil 339/150 \rceil = 3$  replicas 44% faster than HPA, drain-based redistribution (176/163/0 $\rightarrow$ 247/91/0), and all 339 sessions preserved during forced scale-down (step-drop: **2** vs. 639-connection cliff).

## 10. Limitations

- Evaluation environment.** All experiments used a single-machine Kind cluster. Absolute timing values may differ on multi-node production clusters; directional findings are valid.



Figure 14: MQTT-B (`StatefulAutoscaler` with drain): the controller scales on `sum(active_connections)`, reaching 3 replicas at  $t = 38$  s ( $\lceil 339/150 \rceil$ ). Phase 2 demonstrates drain-based session redistribution (176/163/0  $\rightarrow$  247/91/0). Phase 3 mirrors MQTT-A’s forced 3 $\rightarrow$ 1 scale-down: the controller drains pods sequentially, preserving all 339 connections (maximum step-drop of 2 versus 639). Connections reach zero only after the load generator is deleted.

2. **Workload generalizability.** Synthetic linear ramps and deterministic silence gaps were used. Validation against real-world traffic traces with bursty arrivals and variable session durations remains future work.
3. **Observability dependency.** The controller depends on Prometheus freshness; the invariant `scaleDownCooldownSeconds > scrape_interval` must hold. An indefinitely long Prometheus outage will eventually cause the window to expire and scale to `minReplicas`.
4. **No predictive scaling.** The controller reacts to observed connections; proactive pre-warming via `rate(new_connections_total[1m])` is outside the current scope.
5. **Volatile stabilization state.** The sliding window is in-memory; a controller crash resets it. Persisting the high-water-mark to the CR status subresource would eliminate this risk.
6. **Window boundary edge case.** A brief replica dip (5–6) occurs at the CYCLE 1/CYCLE 2 boundary as window entries age out before new entries repopulate. A minimum hold-time following the last scale-up would prevent this.
7. **MQTT limitations.** Drain semantics and per-pod granularity were validated only on

MQTT; WebSocket drain requires end-to-end validation. Both MQTT experiments are single runs (5-run replication is future work). The 339/1,000 connection ceiling is an OS-level Kind environment constraint, not a controller limitation.

8. **Memory instrumentation gap.** The MQTT-B `memory.log` collector recorded zeros; qualitative CPU-independence is unaffected but MQTT memory conclusions are exploratory.

## 11. Future Work

Key open directions include: (i) **OS-level TCP ceiling** – kernel parameter tuning (`net.core.somaxconn`, `tcp_max_syn_backlog`), client-side retry with exponential backoff, headless service addressing, and real cloud cluster validation to confirm whether the 339-client MQTT ceiling is Kind-specific; (ii) **MQTT enhancements** – broker-native session handoff (EMQX, VerneMQ), QoS-aware connection weighting, external session stores for `cleanSession=false` clients, and five-run statistical replication of MQTT-A/B; (iii) **WebSocket extensions** – predictive pre-warming via `rate(new_connections_total[1m])`, drain semantics with protocol-level close frames, CR-status persistence of sliding window state,



Table 5: MQTT extension results: CPU HPA versus `StatefulAutoscaler` with drain semantics. The 339-client ceiling is a transport-level OS limitation present in both runs.

| Metric                        | MQTT-A: CPU HPA           | MQTT-B:<br>StatefulAutoscaler        |
|-------------------------------|---------------------------|--------------------------------------|
| Primary scaling signal        | CPU utilization           | <code>sum(active_connections)</code> |
| Successful connections        | 339 / 1,000               | 339 / 1,000 (same OS ceiling)        |
| First scale-up                | 2 replicas at $t = 68$ s  | 2→3 at $t = 38$ s (44% faster)       |
| Correct replica target        | Never (CPU too low)       | $\lceil 339/150 \rceil = 3$          |
| Per-pod redistribution        | 339/0/0 (sessions pinned) | 176/163/0 → 247/91/0 (drain)         |
| Scale-down method             | Immediate pod termination | Drain-gated (50 s graceful per pod)  |
| Max single-step drop          | 639 (storm to 837)        | 2 (instrument noise only)            |
| Sessions killed by scale-down | ~300                      | 0                                    |
| Peak CPU                      | 110 m                     | 80 m                                 |

per-pod imbalance detection, and real-world traffic trace validation; (iv) **Multi-cluster** – federated Prometheus query patterns and cross-cluster stabilization semantics.

## 12. Conclusion

This paper presented the `StatefulAutoscaler-a` custom Kubernetes controller that resolves the three architectural HPA failure modes for persistent-connection workloads [1]: 87.5% over-provisioning, 1,400 conn/s reconnection storms, and staircase destruction (800→79 sessions). Under Experiment C (five runs), the controller achieved near-exact targeting ( $\lceil 800/100 \rceil = 8$ , overshoot  $\leq +2$  vs. HPA’s 15), zero connection loss, full pod preservation through DROP 1 ( $119 \pm 2$  s scale-down;  $4,280 \pm 67$  pod-seconds), and CPU independence. Scalability validation at 800–3,200 clients confirmed  $\lceil N/T \rceil$  tracking within +2, 66–86 s scale-up, and 100% DROP 1 preservation. Baseline evaluations showed HPA+custom metric and KEDA both preserve pods through DROP 1, with the `StatefulAutoscaler` differentiating on latency (22 s vs. 54 s vs. 26 s) and efficiency (119 s vs. 313 s); critically, baselines succeed only through wide, semantically opaque windows. Three adversarial scenarios bounded the operational envelope. MQTT generalization confirmed protocol-agnosticism (44% faster scale-up, all 339 sessions preserved, step-drop 2 vs. 639-connection cliff). As cloud-native architectures adopt persistent-connection protocols,

connection-aware autoscaling with drain semantics represents a critical infrastructure requirement.

## Data Availability

All source code, experiment scripts, raw results, and analysis scripts are publicly available at <https://github.com/MistaHolmes/ws-k8-controller>. Replication instructions are in `Replication.md`.

## Declaration of Generative AI and AI-Assisted Technologies in the Writing Process

During the preparation of this work, the authors used large language model-based AI tools (Claude, Anthropic) to assist with manuscript drafting, prose refinement, LaTeX formatting, and data analysis scripting. All AI-generated content was critically reviewed, verified, and edited by the authors, who take full responsibility for the accuracy, integrity, and originality of the published work.

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## CRedit Authorship Contribution Statement

**Abhash Behera:** Conceptualization, Methodology, Software, Validation, Formal

Analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization, Project Administration. **Aditya Hota:** Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization. **Suprit Kumar Naik:** Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization. **Ritesh Kumar Singh:** Conceptualization, Methodology, Software, Validation, Formal Analysis, Investigation, Data Curation, Writing – Original Draft, Writing – Review & Editing, Visualization.

## Acknowledgements

The authors gratefully acknowledge Dr. Rakesh Kumar Ranjan for his supervision, strategic input on experimental design and controller implementation, manuscript feedback, and facilitation of institutional compute resources.

## References

- [1] Abhash Behera, Aditya Hota, Suprit Kumar Naik, and Ritesh Kumar Singh. CPU-based autoscaling is harmful for stateful WebSocket workloads: An empirical characterization of Kubernetes HPA failure modes. *Submitted (companion paper)*, 2026. Under review.
- [2] Kubernetes SIG API Machinery. Kubebuilder – SDK for building Kubernetes APIs. <https://book.kubebuilder.io/>, 2024. Accessed: 2026-04-01.
- [3] Kubernetes Authors. Horizontal pod autoscaling – kubernetes documentation. <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>, 2024. Accessed: 2025.
- [4] DirectXMan12. Prometheus adapter for Kubernetes metrics APIs. <https://github.com/kubernetes-sigs/prometheus-adapter>, 2024. Accessed: 2026-04-01.
- [5] KEDA Authors. KEDA: Kubernetes event-driven autoscaling. <https://keda.sh/docs/>, 2024. Accessed: 2024.
- [6] Chenhao Qu, Rodrigo N. Calheiros, and Rajkumar Buyya. Auto-scaling web applications in clouds: A taxonomy and survey. *ACM Computing Surveys*, 51(4):1–33, 2018.
- [7] Fabiana Rossi, Matteo Nardelli, and Valeria Cardellini. Horizontal and vertical scaling of container-based applications using reinforcement learning. *Future Generation Computer Systems*, 109:125–138, 2020.
- [8] Henning Dickel, Vladimir Podolskiy, and Michael Gerndt. Evaluation of autoscaling metrics for (stateful) IoT gateways. In *Proceedings of the IEEE 12th Conference on Service-Oriented Computing and Applications (SOCA)*, pages 17–24. IEEE, 2019.
- [9] Mohamad Imdoukh, Imtiaz Ahmad, and Mohammad Gh Alfalakawi. Machine learning-based auto-scaling for containerized applications. *Neural Computing and Applications*, 32:9745–9760, 2020.
- [10] László Toka, Gábor Dobreff, Balázs Fodor, and Balázs Sonkoly. Machine learning-based scaling management for Kubernetes edge clusters. In *2021 IEEE Conference on Network Function Virtualization and Software Defined Networks*, pages 1–7. IEEE, 2021.
- [11] Zhiheng Zhong and Rajkumar Buyya. Machine learning-based orchestration of containers: A taxonomy and future directions. *ACM Computing Surveys*, 54(10s):1–35, 2020.
- [12] Dipa Soni and Ashwin Makwana. A survey on MQTT: A protocol of Internet of Things (IoT). In *Proceedings of the International Conference on Telecommunication, Power Analysis and Computing Techniques (ICTPACT)*, 2017.
- [13] Andrew Banks and Rahul Gupta. MQTT version 3.1.1. OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>, 2014. Accessed: 2026-05-01.
- [14] Thanh-Tung Nguyen, Yu-Jin Yeom, Taehong Kim, Dae-Heon Park, and Sehan Kim. Horizontal pod autoscaler in Kubernetes for elastic container orchestration. *Sensors*, 20(16):4621, 2020.
- [15] Kubernetes SIG Testing. kind – kubernetes in docker. <https://kind.sigs.k8s.io/>, 2024. Accessed: 2026-04-01.